

Proposition d'architecture

Audit de l'existant · Spécifications techniques

Choix technologiques · Diagrammes UML · Modèle de données

VERSION

2.0

DATE

Avril 2026

AUTEUR

Architecte logiciel

STATUT

Document de référence

i Qu'est-ce qu'un audit technique ?

Une évaluation objective d'un système existant pour comprendre ce qui fonctionne, ce qui pose problème, et pourquoi. L'objectif n'est pas de critiquer les équipes passées, mais d'éviter de répéter les erreurs et de justifier objectivement les décisions.

1.1 Critères d'évaluation

Critère	Définition	Pourquoi c'est important
Maintenabilité	Facilité à corriger, modifier et faire évoluer le code	Un code difficile à maintenir ralentit les équipes et coûte cher
Performance	Vitesse de réponse sous charge normale et en pic	Un site lent = clients qui partent (40 % abandonnent après 2s)
Évolutivité	Capacité à ajouter des fonctionnalités ou supporter plus d'utilisateurs	L'entreprise grandit, le système doit grandir avec elle
Accessibilité	Conformité WCAG 2.1 pour les personnes handicapées	Obligation légale (RGAA) et enjeu éthique
Cohérence fonctionnelle	Homogénéité des fonctionnalités entre marchés	Un client ne doit pas avoir une expérience dégradée selon son pays
Sécurité	Protection des données, conformité RGPD, résistance aux attaques	Obligation légale, confiance des clients

1.2 Description des applications existantes

	Europe (historique)	Amérique du Nord
Backend	PHP 7.x / Symfony	Node.js / Express
Base de données	MySQL 5.7	MongoDB (NoSQL sans schéma)
API exposée	Aucune (monolithe fermé)	Partielle (REST embryonnaire)
Langues	FR, EN, ES, DE	EN uniquement
Fonctionnalités	Complètes : réservation, compte, support email	Limitées : pas de réservation pro, historique 12 mois max

	Europe (historique)	Amérique du Nord
Âge du code	Certaines parties > 10 ans	5 ans — développement rapide sans vision LT
PHP en fin de vie	PHP 7.x hors maintenance depuis déc. 2022	—

1.3 Forces identifiées

Force	Application	À conserver / s'inspirer
Fonctionnalités métier matures	Europe	Processus de réservation rodés, catalogue riche
Support multilingue (4 langues)	Europe	Base de traduction réutilisable pour la migration
API REST partiellement exposée	Amérique du Nord	Bonne pratique à généraliser et formaliser
Authentification établie	Les deux	Les utilisateurs ont des comptes actifs à migrer

1.4 Faiblesses identifiées

Faiblesse	Critère	Impact concret
Deux codebases séparées	Maintenabilité	Toute évolution développée deux fois. Bugs corrigés dans une app peuvent persister dans l'autre.
Aucun compte cross-marchés	Cohérence	Un client européen au Canada ne voit pas ses réservations européennes.
PHP 7.x hors maintenance	Sécurité	Aucun correctif de sécurité depuis décembre 2022. Failles non corrigées.
MongoDB sans schéma strict	Maintenabilité	Données incohérentes (champs nommés différemment selon l'époque), migrations instables.
Technologies hétérogènes	Maintenabilité	PHP + Node.js : compétences différentes, recrutement complexe, coûts doublés.
Aucun audit accessibilité	Accessibilité	Non-conformité RGAA probable → risque juridique + exclusion des PSH.
Aucune API côté Europe	Évolutivité	Impossible de connecter une app mobile native sans refonte complète.
Pas de logs centralisés	Sécurité	Traçabilité insuffisante pour le RGPD et la détection d'intrusions.

1.5 Conclusion de l'audit

Critère	Résultat	Synthèse
Maintenabilité	✗ Insuffisante	Deux codebases, technologies hétérogènes, PHP en fin de vie
Performance	⚠ Non mesurée	Acceptable mais aucun indicateur documenté
Évolutivité	✗ Faible	Architectures fermées, pas d'API Europe, pas de tchat
Accessibilité	✗ Non conforme	Aucun audit WCAG réalisé, risque légal avéré
Cohérence fonctionnelle	✗ Insuffisante	Expérience très différente selon le marché
Sécurité	⚠ Partielle	Authentification en place, mais PHP vulnérable et logs absents

✓ Conclusion

La décision de concevoir une nouvelle application centralisée est **techniquement justifiée et nécessaire**. Continuer à maintenir l'existant représenterait un risque croissant (sécurité, légal, coûts) sans améliorer l'expérience client.

2.1 Contraintes héritées de l'existant

Contrainte	Explication	Impact sur le projet
Migration progressive	L'application Europe reste opérationnelle pendant la transition (milliers de clients actifs)	Les deux systèmes coexistent 6 à 12 mois — pas de "big bang"
Migration des données	Clients et réservations existants à transférer	Plan de migration ETL dédié à prévoir (hors périmètre PoC)
Zéro downtime	Service critique pour voyageurs en déplacement	Stratégie de déploiement sans interruption

2.2 Exigences non fonctionnelles techniques (mesurables)

Exigence	Valeur cible	Outil de mesure
Temps de réponse HTTP	< 2 secondes (95e percentile)	Tests de charge avec Locust ou k6
Latence WebSocket (tchat)	< 500 millisecondes	Mesure round-trip time en test
Disponibilité mensuelle	≥ 99,5 % (≈ 3h30 d'arrêt max/mois)	Monitoring Uptime Robot / Datadog
Conformité accessibilité	WCAG 2.1 niveau AA	Audit Axe DevTools + tests NVDA
Conformité RGPD	Droits access / rectif / effacement / portabilité	Revue juridique + tests fonctionnels
Langues supportées	FR, EN, ES (extensible)	Tests de non-régression des traductions
Sécurité	OWASP Top 10 couvert	Audit de sécurité (pentest léger) avant mise en prod

2.3 Contraintes de déploiement

Contrainte	Décision	Justification
Hébergement	Cloud (AWS ou GCP)	Flexibilité, scalabilité, pas d'infrastructure physique
Conteneurisation	Docker recommandé	Reproductibilité des environnements (dev = recette = prod)

Contrainte	Décision	Justification
Environnements	Développement → Recette → Production	Tester avant de déployer, jamais directement en prod
Chiffrement	HTTPS et WSS obligatoires en production	Exigence RGPD + bonne pratique universelle
Sauvegardes	Quotidienne, conservation 7 jours	Garantie contre la perte de données

3.1 Comparaison des modèles d'architecture

✗ Écarté

Microservices

- + Scalabilité granulaire par service
- + Pannes isolées
- Complexité opérationnelle très élevée
- Nécessite une équipe DevOps dédiée
- Coût infrastructure doublé

✗ Écarté

Serverless

- + Pas de serveur à gérer
- + Paiement à l'usage
- WebSocket très complexe en serverless
- Vendor lock-in fort
- "Cold start" (latence au démarrage)

✓ Retenu

Monolithe modulaire

- + Déploiement simple
- + Cohérence des données
- + Équipe unique, outillage unifié
- + Scalable horizontalement
- Scalabilité moins granulaire qu'en microservices



Analogie

Le monolithe modulaire, c'est comme un immeuble de bureaux bien organisé en départements (RH, comptabilité, commercial...) plutôt qu'une ville de micro-appartements éparpillés. Tout est au même endroit, organisé, mais chaque département a ses responsabilités.

3.2 Architecture en couches + couche événementielle

COUCHE PRÉSENTATION

Templates HTML · API REST (DRF) · WebSocket (Channels)
Ce que l'utilisateur voit et avec quoi il interagit

COUCHE LOGIQUE MÉTIER

Apps Django : accounts · vehicles · reservations · chat
Les règles de l'entreprise (ex : annulation -48h)

COUCHE DONNÉES

ORM Django + PostgreSQL
Stockage et récupération des informations



Couche événementielle (tchat temps réel)

Django Channels ↔ Redis Channel Layer ↔ WebSocket

3.3 Rôle de chaque composant

Composant	Rôle technique	Analogie
Nginx	Reverse proxy : reçoit toutes les connexions, gère TLS, cache statiques, distribue la charge	Le portier d'un immeuble qui oriente les visiteurs

Composant	Rôle technique	Analogie
Daphne	Serveur ASGI : exécute Django, gère HTTP <i>et</i> WebSocket simultanément	Les employés qui traitent les demandes
Django	Framework MVC : logique métier, vues, authentification, ORM, admin	Le système de règles de l'entreprise
Django Channels	Extension Django pour les WebSockets — gère les connexions persistantes du tchat	Le téléphoniste qui maintient les lignes ouvertes
Redis	Channel Layer : synchronise les messages WebSocket entre plusieurs workers Daphne	Le standard téléphonique central
PostgreSQL	Base de données relationnelle : stocke toutes les données de l'application	L'armoire à dossiers de l'entreprise



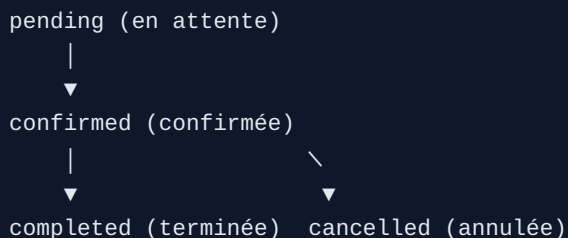
WebSocket vs HTTP classique

HTTP classique : le client envoie une requête → le serveur répond → la connexion est fermée. Comme envoyer une lettre et attendre la réponse par courrier.

WebSocket : connexion permanente, les deux parties peuvent s'envoyer des messages à tout moment. Comme un appel téléphonique : la ligne reste ouverte.

Entité	Champs principaux	Décisions de conception notables
User	id, email (unique), first_name, last_name, language, country, is_active, is_staff, date_joined	Hérite de <code>AbstractUser</code> Django — authentification intégrée, pas de réimplémentation
Room	id, name (unique), created_at, participants (M2M → User)	<code>participants</code> en Many-to-Many : une room peut avoir plusieurs agents, un utilisateur peut avoir plusieurs rooms
Message	id, room (FK), author (FK nullable), content, sent_at, is_read	RGPD : <code>author</code> est nullable (<code>SET_NULL</code>). À la suppression du compte, l'auteur devient NULL — le message est conservé mais anonymisé
Vehicle	id, brand, model, year, category, seats, transmission, fuel, price_per_day, is_available, agency (FK)	<code>is_available</code> permet de masquer un véhicule en maintenance sans le supprimer
Agency	id, name, address, country, latitude, longitude, phone, is_active	<code>is_active = False</code> : désactivation logique sans suppression (historique des réservations passées conservé)
Reservation	id, client (FK → User), vehicle (FK → Vehicle), start_date, end_date, pickup_agency, return_agency, status, total_price, created_at	<code>status</code> avec choices : <code>pending</code> → <code>confirmed</code> → <code>completed</code> ou <code>cancelled</code>

4.1 Cycle de vie d'une réservation



4.2 Relations entre entités

Relation	Type	Signification
User ↔ Room	Many-to-Many	Un utilisateur participe à plusieurs rooms, une room peut avoir plusieurs participants
Room → Message	One-to-Many	Une room contient plusieurs messages

Relation	Type	Signification
User → Message	One-to-Many (nullable)	Un utilisateur envoie plusieurs messages ; à la suppression, author = NULL
Agency → Vehicle	One-to-Many	Une agence possède plusieurs véhicules
User → Reservation	One-to-Many	Un client peut avoir plusieurs réservations
Vehicle → Reservation	One-to-Many	Un véhicule peut avoir plusieurs réservations (historique)

5.1 Framework backend — Django 4.2 LTS

Critère	Django 4.2 ✓	FastAPI	Node.js / Express
Maturité	• 20 ans d'existence	Créé en 2018	15 ans
ORM intégré	✓ Oui	✗ Non (SQLAlchemy externe)	✗ Non (Sequelize externe)
Admin back-office	✓ Auto-généré	✗ À développer	✗ À développer
Authentification	✓ Intégrée	✗ Manuel	~ Bibliothèques tierces
i18n multilingue	✓ Natif	~ Partiel	✗ Externe
WebSocket (tchat)	✓ Via Channels	~ Natif mais limité	✓ Natif
Support LTS	✓ Jusqu'en avril 2026	✗ Pas de LTS	—

✓ Décision : Django 4.2 LTS retenu

Le back-office admin auto-généré représente à lui seul plusieurs semaines de développement économisées. L'ORM évite le SQL manuel et réduit les erreurs. L'authentification et l'i18n intégrées sont alignées avec les exigences du projet. Le LTS garantit la sécurité jusqu'en 2026.

5.2 Temps réel — Django Channels 4 + Redis

Technologie	Fonctionnement	Adapté pour le tchat ?
WebSocket (retenu)	Connexion bidirectionnelle persistante	✓ Oui — la meilleure option
Server-Sent Events	Connexion unidirectionnelle (serveur → client seulement)	~ Non — le client ne peut pas envoyer
Long polling	Le client redemande toutes les N secondes	✗ Non — fausse impression, charge serveur élevée

Critère	Django Channels + Redis ✓	Socket.io (Node.js)	Pusher (SaaS)
Intégration Django	✓ Native officielle	✗ Aucune	~ Via SDK

Critère	Django Channels + Redis ✓	Socket.io (Node.js)	Pusher (SaaS)
Second service à maintenir	✓ Non	✗ Oui (Node.js)	✗ Oui (dépendance externe)
Données transitant chez un tiers	✓ Non	✓ Non	✗ Oui (RGPD !)
Coût	Open source gratuit	Open source gratuit	✗ Abonnement mensuel

5.3 Base de données — PostgreSQL 15

Critère	PostgreSQL ✓	MySQL	MongoDB
Schéma strict	✓ Oui	✓ Oui	✗ Non (problème audit)
Transactions ACID	✓ Complètes	✓ Complètes	~ Partielles (v4+)
Support Django ORM	✓ Natif premium	✓ Natif	✗ Via djongo (non officiel)
Extensions utiles	✓ PostGIS, Full-text	~ Limitées	—

✓ Décision : PostgreSQL 15 retenu

L'audit a démontré les problèmes concrets de MongoDB (incohérences de données). PostgreSQL garantit un schéma strict, des transactions ACID (critiques pour les réservations), et un support Django natif. PostGIS permet d'ajouter la géolocalisation des agences sans changer de base.

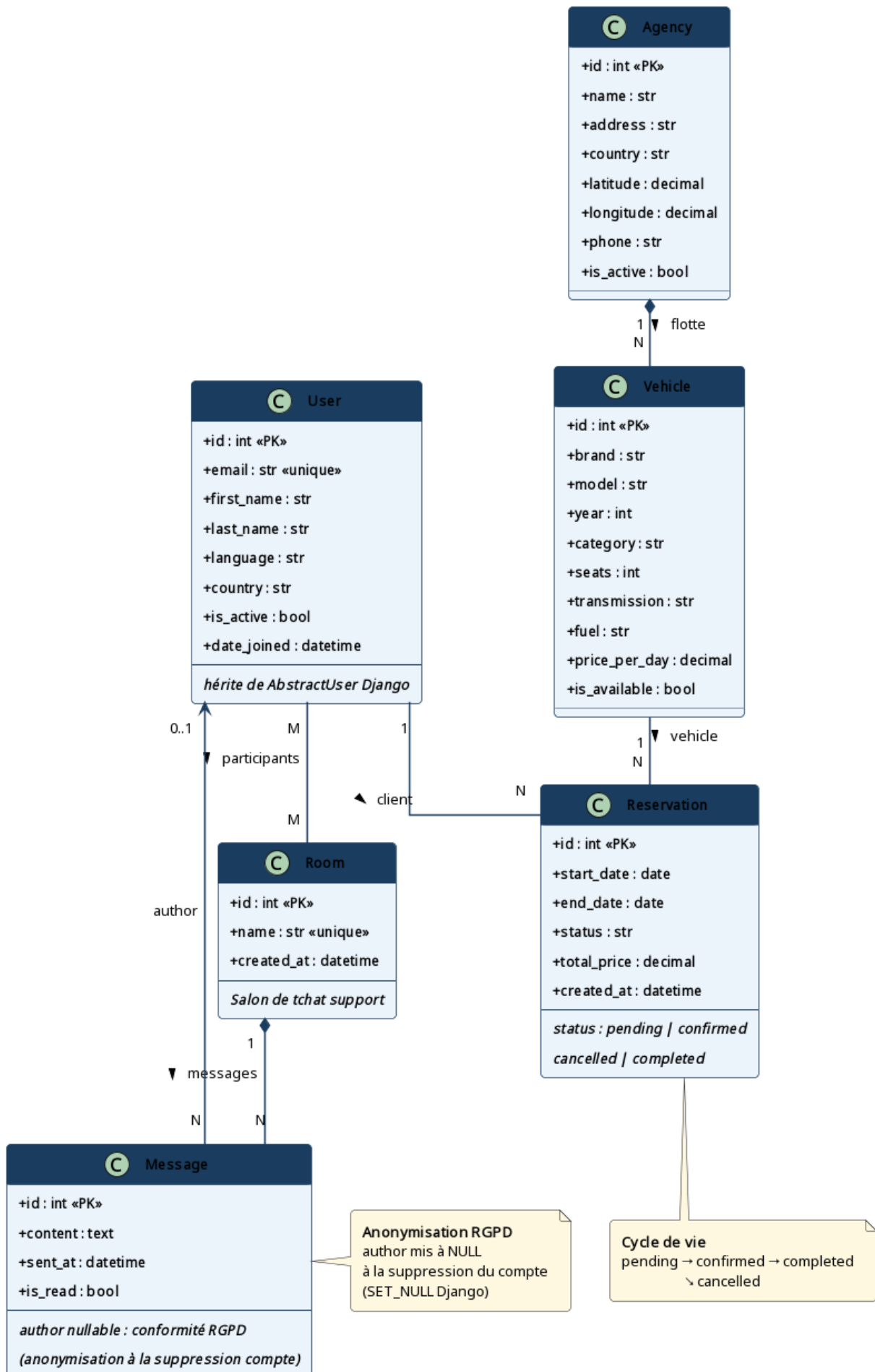
5.4 Serveur ASGI — Daphne

⚠ Pourquoi pas Gunicorn ?

Gunicorn est le serveur classique de Django (WSGI). Mais WSGI est **synchrone** et ne supporte pas les WebSockets. Dès lors qu'on intègre Django Channels pour le tchat, on doit passer à **ASGI**. Daphne est le serveur ASGI officiel de l'équipe Django Channels — zéro configuration supplémentaire.

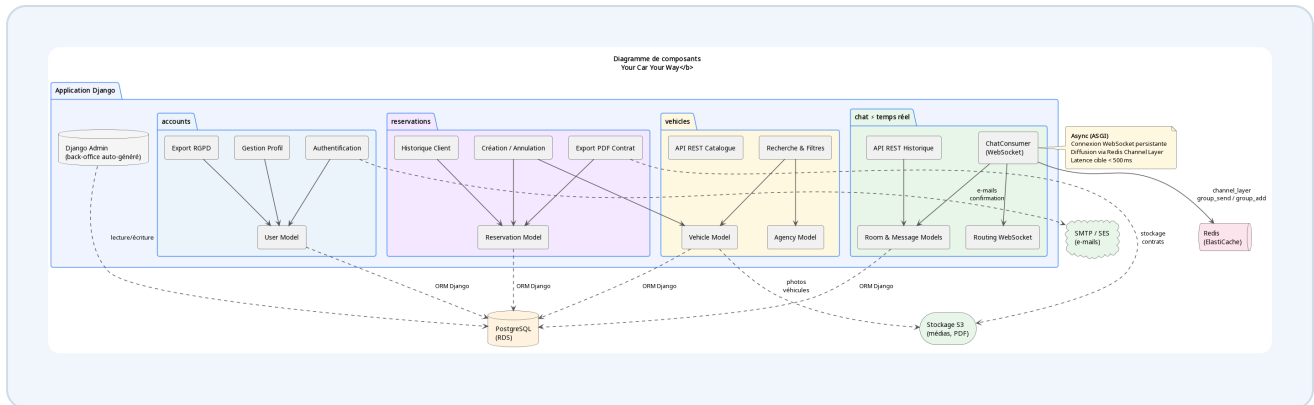
Représente les entités persistées en base de données (User, Room, Message, Vehicle, Agency, Reservation), leurs attributs et leurs relations. C'est la vue "données" de l'application.

Diagramme de classes — Modèle de données
Your Car Your Way



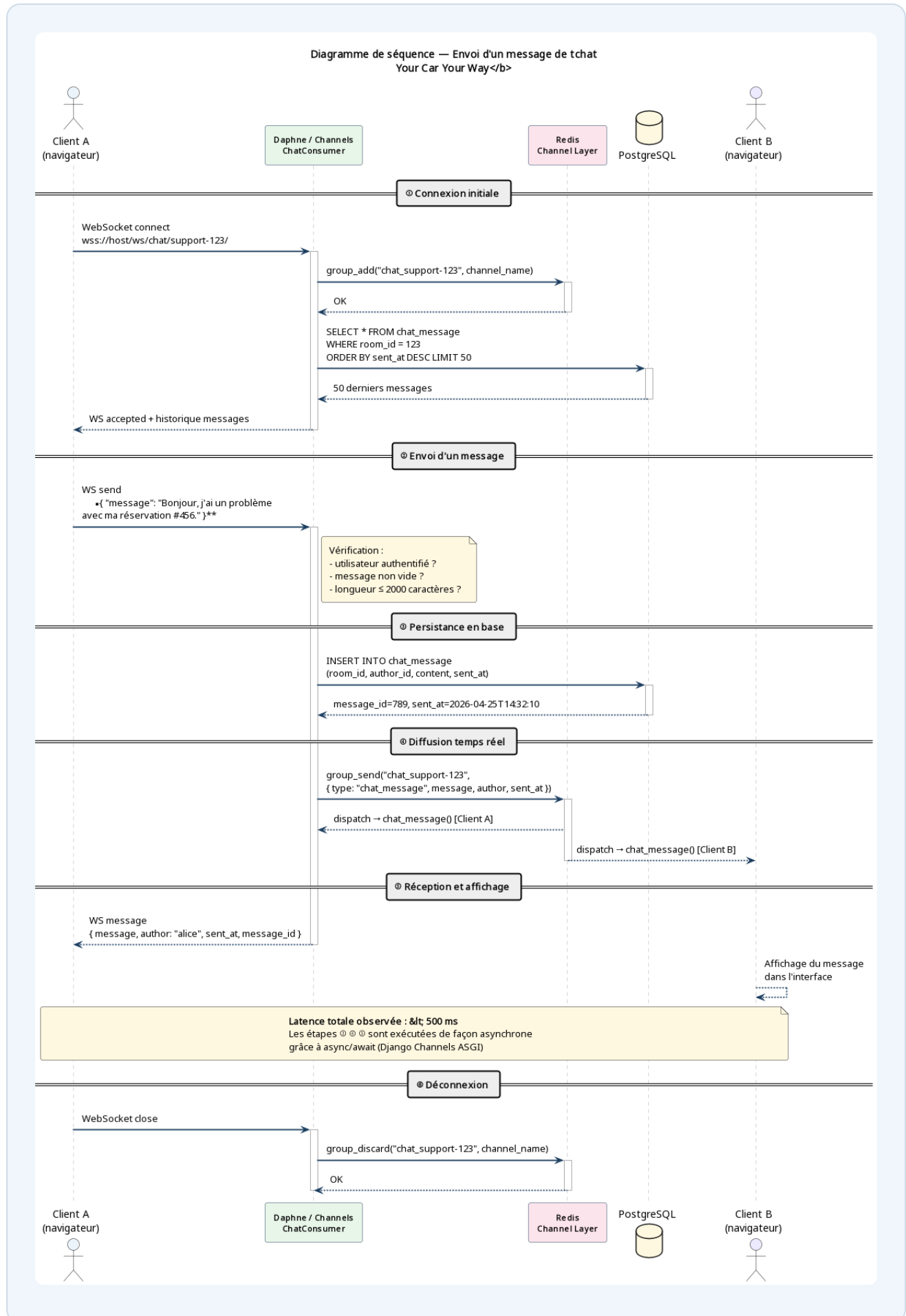
6.2 — DIAGRAMME DE COMPOSANTS · ORGANISATION DU CODE

Représente les modules applicatifs Django (accounts, vehicles, reservations, chat) et leurs dépendances vers l'infrastructure externe (PostgreSQL, Redis, S3, SES). C'est la vue "organisation du code".



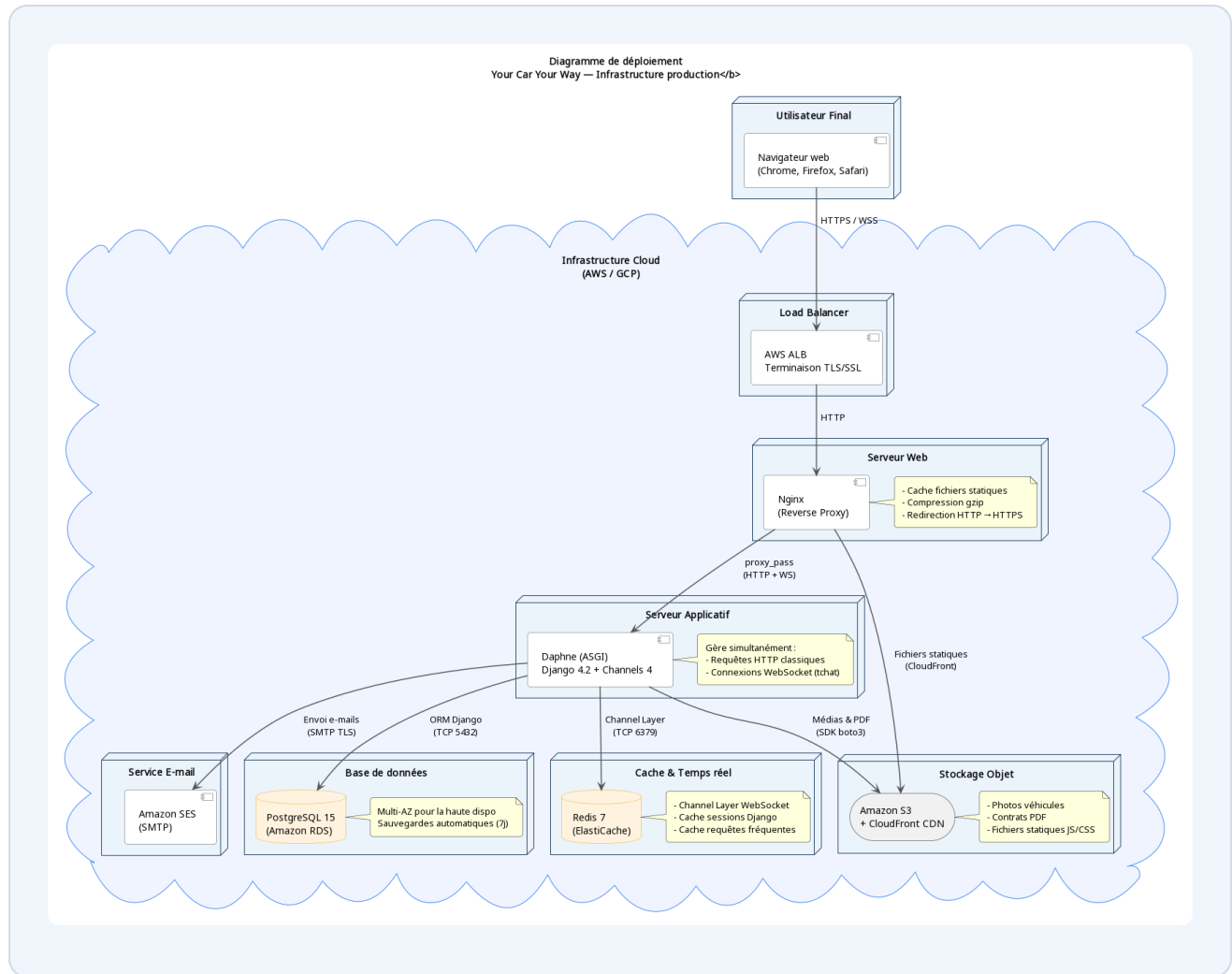
6.3 — DIAGRAMME DE SÉQUENCE · ENVOI D'UN MESSAGE DE TCHAT

Chaque colonne est un acteur ou composant. Les flèches sont les échanges dans l'ordre chronologique (temps de haut en bas). Illustre les 6 étapes : connexion → envoi → persistance → diffusion Redis → réception → affichage.



6.4 — DIAGRAMME DE DÉPLOIEMENT · INFRASTRUCTURE CLOUD

Représente le déploiement en production : Nginx (reverse proxy TLS) → Daphne (ASGI) → PostgreSQL / Redis / S3 / SES. Architecture cloud avec haute disponibilité et sauvegardes automatiques.



7.1 Tableau récapitulatif

Décision	Choix retenu	Alternative écartée	Raison principale
Architecture globale	Monolithe modulaire	Microservices	Complexité adaptée à l'équipe, évolutif si besoin
Framework backend	Django 4.2 LTS	FastAPI, Node.js	ORM, admin auto, authentification, i18n, LTS
Temps réel	Django Channels + Redis	Socket.io	Intégration Django native, pas de second service
Base de données	PostgreSQL 15	MongoDB	Schéma strict, ACID, support ORM natif
Serveur applicatif	Daphne (ASGI)	Gunicorn (WSGI)	Gunicorn ne supporte pas WebSocket
API	REST (DRF)	GraphQL	Standard, outillage mature, besoins bien définis
Proxy / TLS	Nginx	Apache	Performance, cache statiques, config WebSocket
Langage	Python 3.11+	PHP, JavaScript	Unification sur un langage, écosystème IA/data

7.2 Cohérence avec les exigences fonctionnelles

Exigence fonctionnelle	Réponse technique
Compte unique cross-marchés	Base PostgreSQL unique, modèle <code>User</code> centralisé
Tchat temps réel (< 500 ms)	Django Channels + WebSocket + Redis Channel Layer
Accessibilité WCAG 2.1 AA	HTML sémantique dans les templates, <code>aria-live</code> sur le tchat, <code>role="log"</code>
Multilingue (FR, EN, ES)	Système i18n natif Django (fichiers <code>.po</code> / gettext)
RGPD — droit à l'effacement	Champ <code>author</code> nullable (<code>SET_NULL</code>), endpoint d'export JSON
Performance < 2 secondes	Nginx cache statiques, PostgreSQL indexé, Redis cache sessions
Back-office administrateur	Django Admin auto-généré, personnalisé avec <code>ModelAdmin</code>

Exigence fonctionnelle	Réponse technique
API pour futures apps mobiles	Django REST Framework — endpoints JSON documentés

7.3 Limites et évolutions futures (honnêteté)

Limite actuelle	Quand devient problématique	Solution envisageable
Scalabilité du monolithe	Trafic > 10 000 utilisateurs simultanés	Extraction en microservices des modules les plus sollicités
Base de données unique	Volume de données très important (millions de messages)	Partitionnement PostgreSQL ou base séparée pour le tchat
Pas de paiement en ligne	Dès que requis par le business	Intégration Stripe ou Adyen en Phase 2
Pas d'app mobile native	Si les utilisateurs mobiles deviennent majoritaires	API DRF déjà prête — développement React Native / Flutter en Phase 3